EEE DOMAIN S2 Report

OBJECTIVES:

The electrical and electronic engineering (EEE) section of a boat project typically covers the design, installation, and integration of all the electrical systems and electronic components on the vessel.

During the first semester, as part of the electrotechnical boat project, electronic and automatic (EEE), the deliverable was to Break down the motor control system into functional blocks, design and simulate the Analogic functional blocks model.

Now during this second semester we are going to focus on the Breakdown of the digital part of the motor control into functional blocks, design and simulate the blocks using suitable software and test them on suitable hardware.

Action Plan

- Objective and explanation of contents
- Work principle and role of each component
- Controlling the speed rotation couple of the motor
- Information chain and circuit supply chain
- Digital functional blocks synthesis diagram representation circuit

WORK PRINCIPLE AND ROLE OF EACH COMPONENTS

1- DC MOTOR

A DC motor is an electrical motor that uses direct current (DC) to produce mechanical force.The most common types rely on magnetic forces produced by currents in the coils. Nearly all types of DC motors have some internal mechanism, either electromechanical or electronic, to periodically change the direction of current in part of the motor.

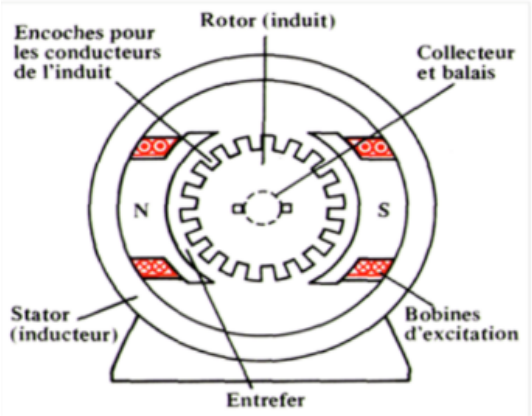


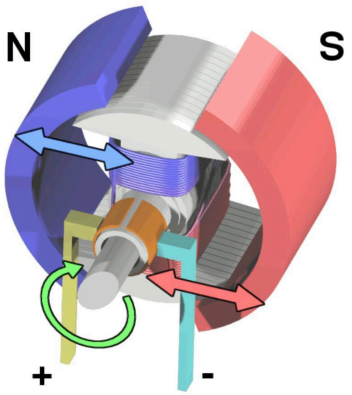
ATTRIBUT	VALUE
Supply voltage	12 Vcc
Power output	14W
Output speed	4289 tr/min
Shaft diameter	6 mm
Maximum output torque	350 gcm
Length	133.5mm
Dimensions	(∅) 37 mm
Current	11.5 A
Core material	Steel Core
Weight	0.75 pounds



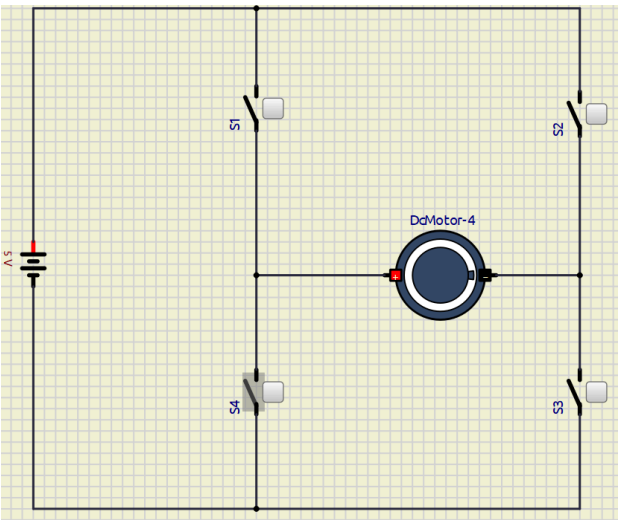
Principle of Dc motor:

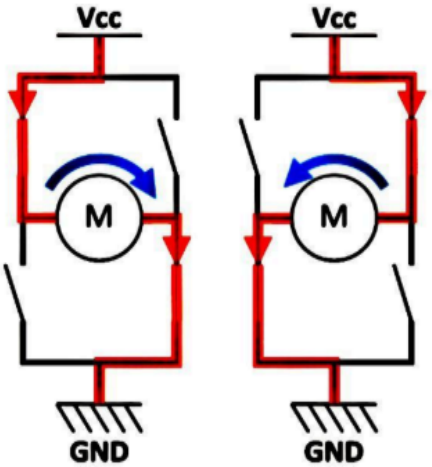
A DC motor is made up of a magnetic circuit featuring a fixed part named stator, a rotative part named rotor and air-gap between both parts:The inductor (stator) creates fields which rotate by winding.An induced electrical circuit (rotor) is affected by this magnetic field. The collector and brushes provide access to the rotor circuit.

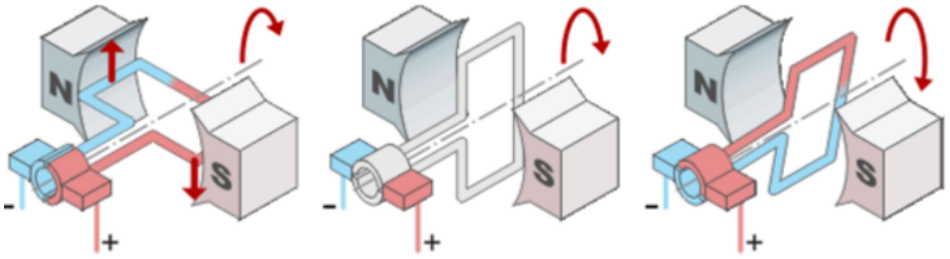




Controlling a motor to turn in 2 directions using switches, We observe that when S1 and S3 are closed(connected) while S2 and S4 are opened the motor turns towards the right direction and vice versa. It just function as an H-Bridge







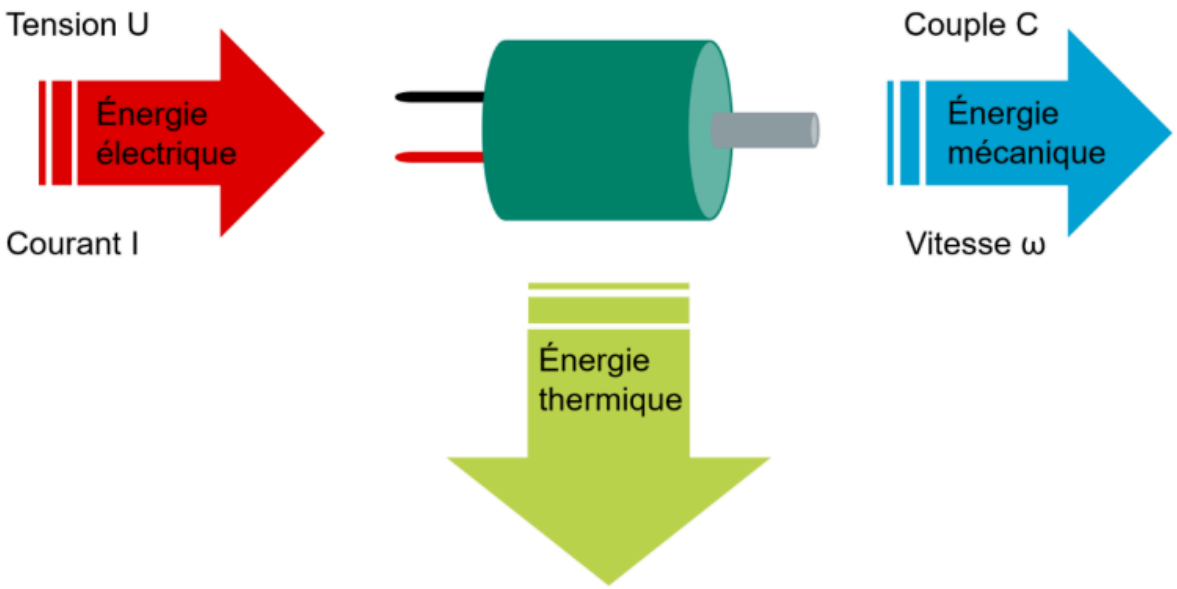
Advantages of dc motor

1. DC motors have simpler installation and maintenance.
2. DC motors are typically more efficient and make better use of their input energy.
3. They have high startup power
4. They have high starting torque; it's as high as 500% compared to normal operating torque.
5. They have fast response times to starting, stopping, and acceleration.

Disadvantages of dc motors

1. High initial cost.
2. Its operation cost and maintenance cost is very high due to the presence of commutators and brush gear.

TECHNICAL NOTIONS

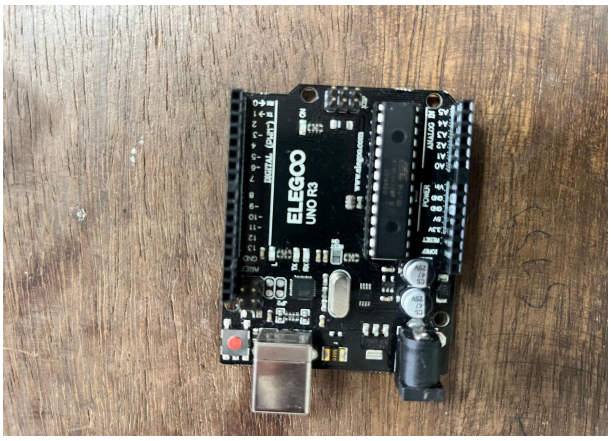


Electrical power (current/voltage) is converted into mechanical power (speed/torque) by the electric motor. The losses associated with this transformation are losses due to friction (mechanical) and heat dissipation (Joule losses) in the winding. For brushless motors, Joule losses are combined with friction losses.

2- Arduino microcontroller board

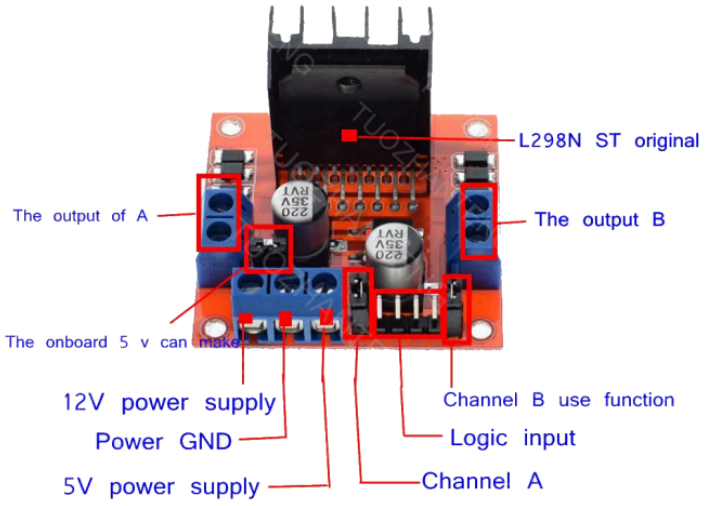
Arduino senses the environment by receiving input from many sensors, and affects its surroundings by controlling lights, motors, and other actuators.

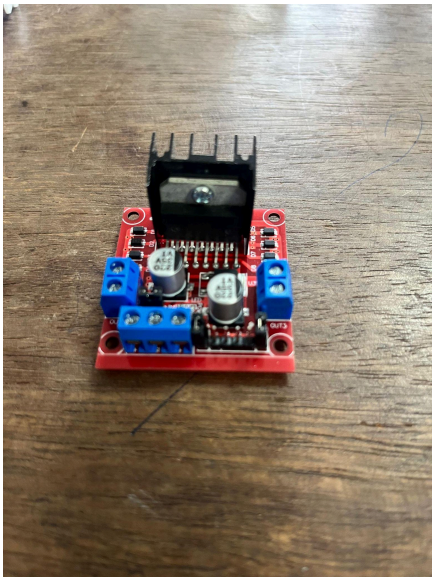




3- **DC Motor Driver(H-Bridge)**

The Dual DC Motor Driver 4A V3 allows two DC motors to be driven independently, controlling both speed and direction. The H-bridge receives the PWM signal as input and regulates the quantity of energy supplied to the motor.





4- Power supply

It's essential to have an adequate source of energy to provide the voltage and current required for the operation of the H-bridge and the motor. This source can be a battery, a regulated power supply or any other source suitable for energy.

5- Button

We've included a button to change the direction of rotation of our DC motor

6- Potentiometer

A user-adjustable 3 terminal variable resistor that can be used as a voltage divider. The first one will be used as a dimmer for the LCD screen, the second one will be used to set the boat direction.

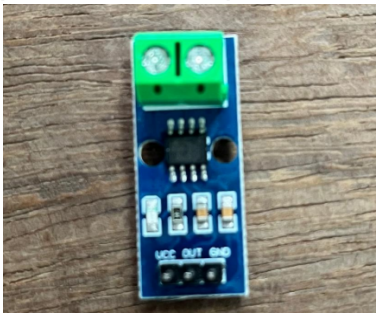


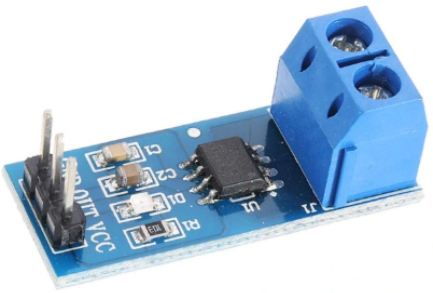


7-Current and Speed Sensors

Speed sensor measures the rotational speed of the motor by detecting the movement of the motor shaft and generating a corresponding output signal, allowing the speed to be calculated.

Current sensor using a sensor effect Hall, this device measures the current consumed by the motor. This information is essential for monitoring power consumption and preventing overcurrent.





1. Control System Breakdown

· **External Data:**

This refers to instructions sent by a user or a higher-level system: desired speed, direction, operating mode, etc. It can come from a remote control, a setpoint sensor, or a human-machine interface (HMI).

- **Conversion:**

This block transforms physical (analog) data from sensors (e.g., voltage, current, speed) into digital signals that can be used by the control board. It is an analog-to-digital (A/D) interface. It also supplies the sensors with current (VDC).

- **Current sensor:**

This measures the electrical current drawn by the motor. This allows you to monitor the motor's load and detect potential overloads or mechanical blockages.

- **Speed sensor:**

This provides the motor's actual rotation speed. This data is essential for comparing it with the setpoint and regulating the motor.

- **Digital board:**

This is the heart of the control system. It can be a microcontroller, an FPGA, or an Arduino. It executes a control program that:

- Reads data from the sensors via conversion.
- Compares the measurements with the setpoints.
- Calculates a command (often PWM) to adjust the motor.

- **M (Motor):**

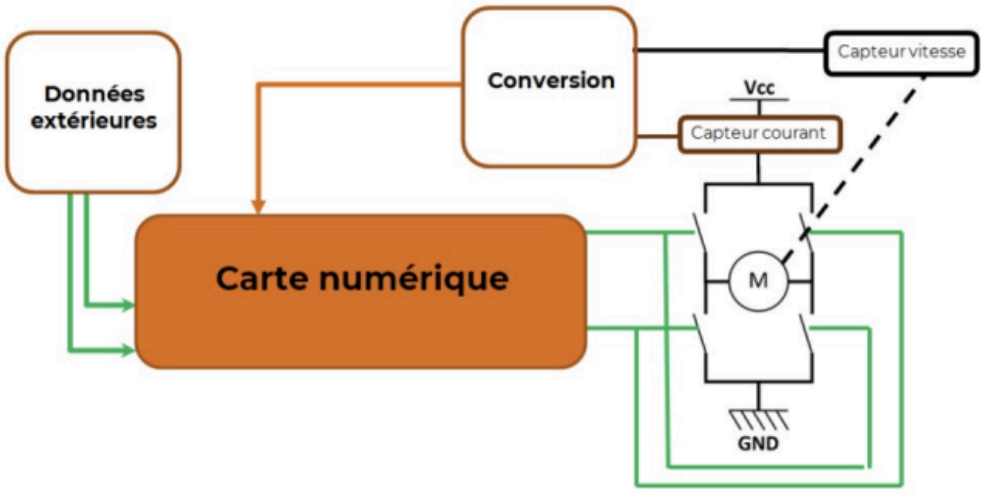
This is the final actuator. It receives power via PWM or current and executes a movement (rotation, propulsion, etc.).

· GND and Vcc:

Ground (GND) and power supply (Vcc) for the system.

Overall system operation:

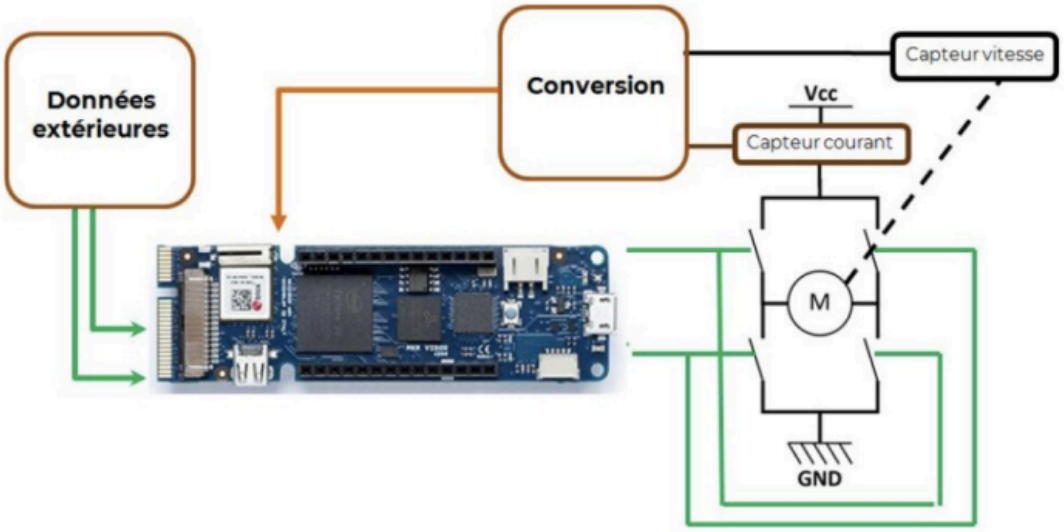
1. The user (or another machine) sends a command via external data.
2. The digital board receives this data, then reads the actual state of the motor using the sensors.
3. The conversion transforms the physical signals from the sensors into usable digital values.
4. The board compares the command and the actual state and calculates a corrective command.
5. This command is sent to the motor to correct its speed or direction.
6. The loop repeats in real time: it is a closed-loop system.



Interactions in the command part:

- The digital board relies on sensors to determine the motor's status.
- The sensors are powered and monitored by the conversion unit, which plays a critical role.
- Any measurement error (bad sensor or poor conversion) can distort the control.
- The motor is controlled by a PWM signal modulated according to the board's internal calculations.

2. Automated system (with Arduino)



This diagram uses the same blocks as the previous one, but with a concrete application using an Arduino card as the heart of the system.

Central component: Arduino

Arduino is an open-source microcontroller board widely used in educational and semi-professional embedded systems.

· It:

- Reads values from sensors (via analog or digital pins).
- Executes a control program (e.g., PID).
- Generates a PWM signal to the motor.

Can also communicate with a PC, radio, or another Arduino.

Special feature of this diagram:

· Concrete visualization: Here, the digital board is explicitly represented by an Arduino, which allows for a link between theory and physical reality.

Visible physical interactions: Connections between the blocks are made via wires connected to specific pins (Vcc, GND, PWM, analog inputs, etc.)

Conclusion common to both diagrams:

·These diagrams show a modular, feedback, and intelligent architecture where:

- Sensors enable real-time monitoring of the motor.
- The board (Arduino or FPGA) enables dynamic control based on algorithms (PID, fuzzy logic, etc.).

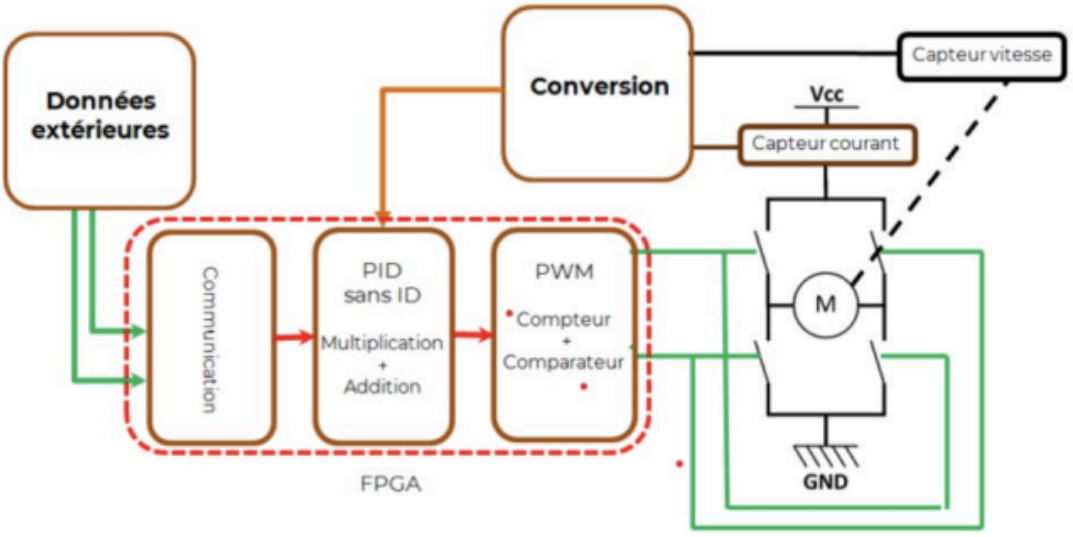
The motor executes the action and loops the information back via the sensors.

3. FPGA-based control loop with motor and current sensor

- FPGA (Field Programmable Gate Array): Heart of the processing system.

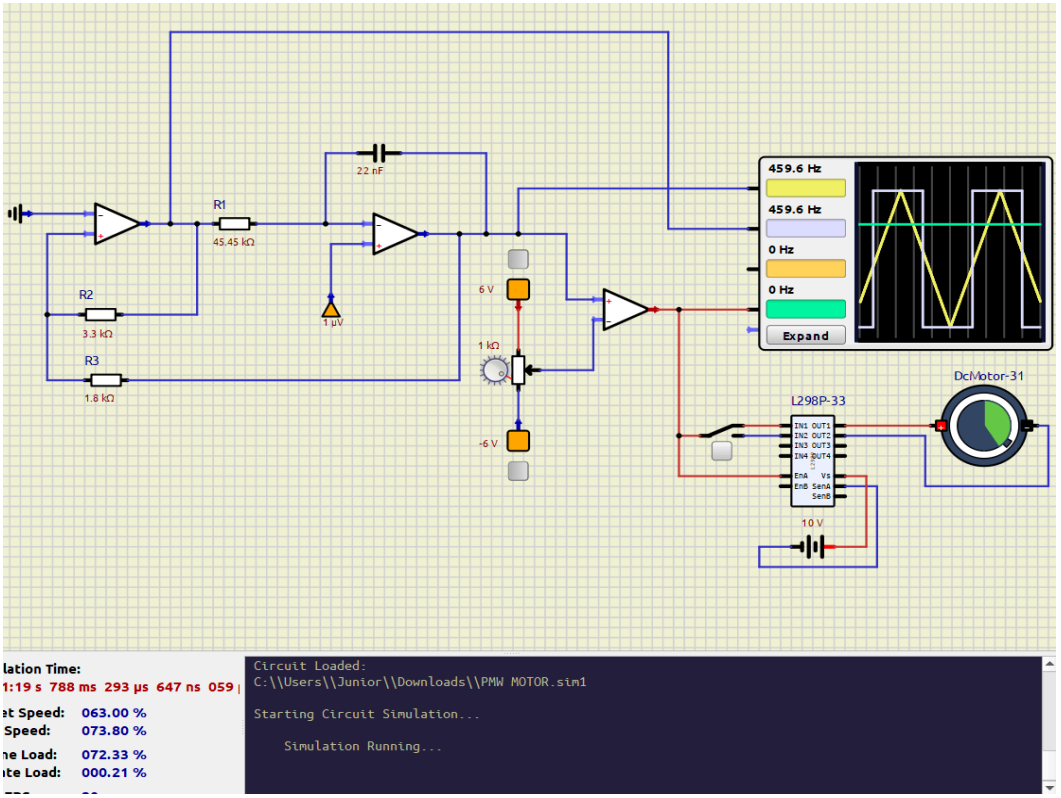
❖ Communication: Receives and exchanges data with the environment (Arduino, radio, etc.).
❖ ID-less PID: Proportional + Derivative (PD) controller. It adjusts the action on the motor based on the difference between the desired and actual speed, but without integration.
❖ Multiplication / Addition: Internal calculations within the PID to obtain the correction command.
❖ PWM (Pulse Width Modulation): Generates a square waveform whose duty cycle controls the motor's power.
❖ Counter: Generates a time base.
❖ Comparator: Compares the setpoint with the time base to create the PWM signal.

- M (Motor): Executes the command (speed or direction).



Overall operation:

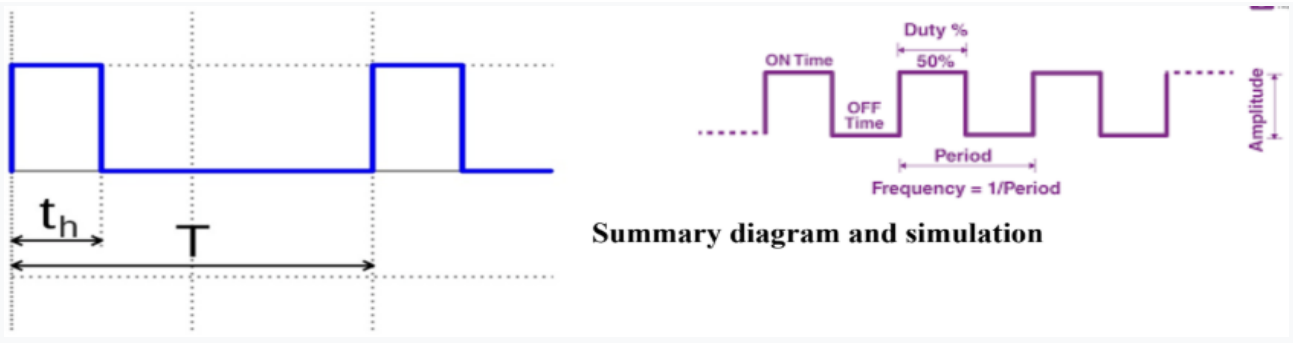
1. The system receives a speed command.
2. The current sensor verifies the force applied by the motor.
3. The FPGA dynamically adjusts the PWM based on the measurement using its PD controller.
4. The motor reacts to the PWM, changing its speed.



PMW SIGNAL(PULSE WIDTH MODULE)

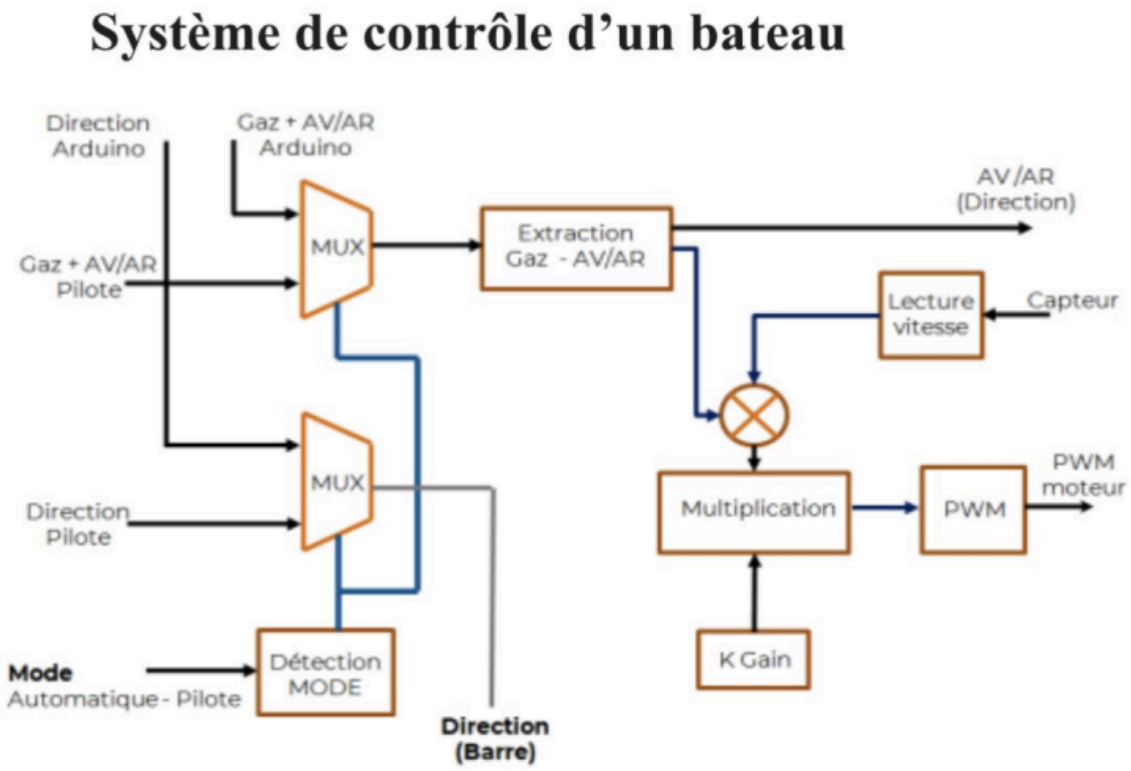
PWM (Pulse Width Modulation) is a digital signal, therefore the voltage can only take two values. The signal is square. The low level generally corresponds to 0 Volts. The period is noted as T; the duration of the pulse (during which the voltage is that of the high state) is called th. More precisely, Pulse Width Modulation "PWM" provides a rectangular logic signal.

All the information resides in the duty cycle, which designates the duration of the high state during a period T. PWM varies the duration of the high state (which we will call th). This changes the duty cycle (the duty cycle is expressed as th/T and can vary from 0 to 1).



5. The loop starts again continuously.

4. Boat directional control system and engine



Components and Functions:

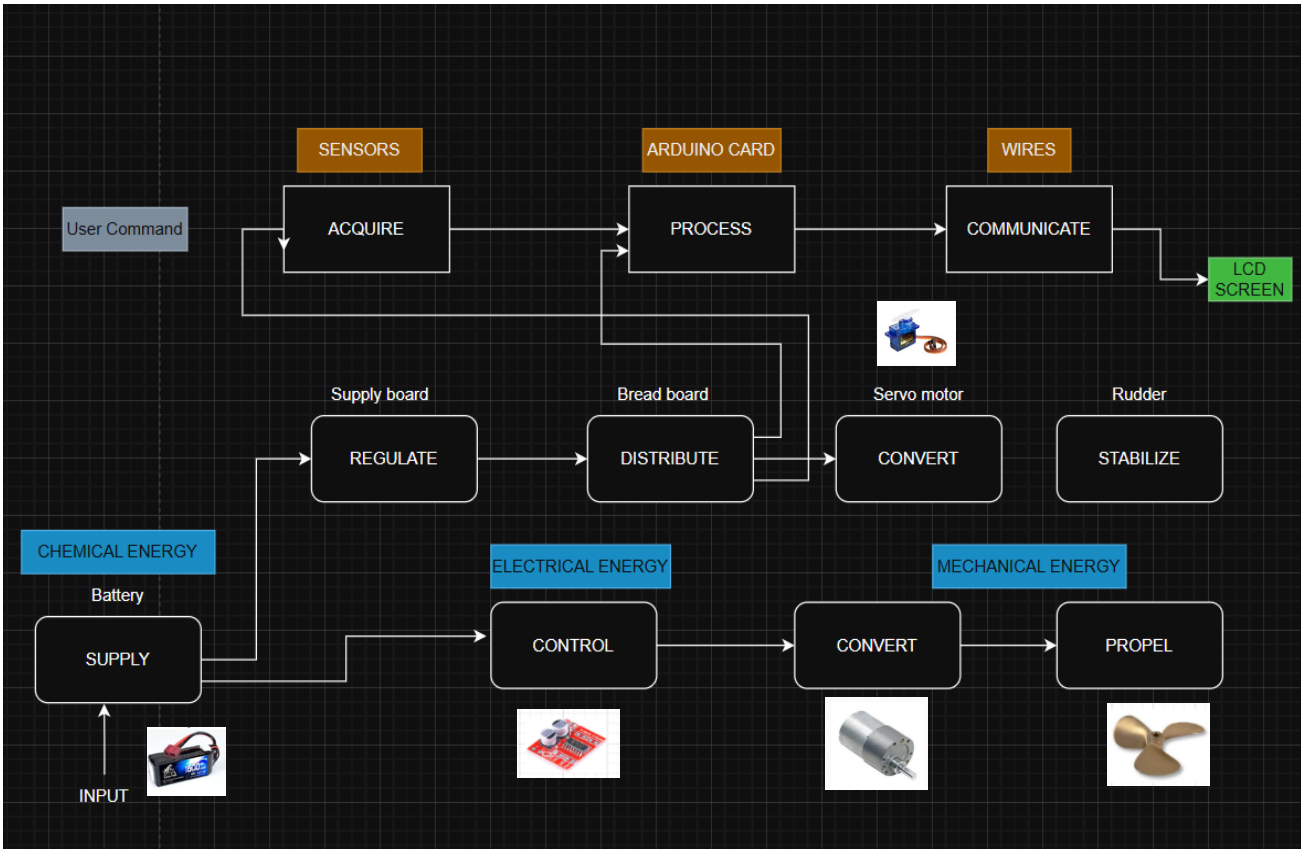
- **MUX (Multiplexer):**
    - Selects one of the two control sources: **Arduino (automatic mode)** or **Pilot (manual mode)**.
    - There are two multiplexers: one for **Throttle control** (forward/reverse) and one for **Steering**.

- **MODE Detection:**
    - Reads the state of the switch (manual or automatic) and configures the multiplexers accordingly.
- **Throttle Extraction – Forward/Reverse:**
    - Interprets the throttle command to determine whether it indicates **forward motion** or **reverse**.
    - Sends this information to the next part of the system.
- **Speed Sensor (Speed Reading):**
    - Provides the current speed of the boat.
    - Allows for automatic adjustment through **error calculation**.
- **Multiplication:**
    - Performs a **proportional calculation** between the speed error and an **adjustment factor (K Gain)**.
- **K Gain:**
    - A **correction constant** that determines the sensitivity of the system.
    - For example, if the boat is going too fast or too slow, this factor defines how much to correct the speed.
- **Motor PWM:**
    - A **Pulse-Width Modulated signal** (variable duty cycle) used to control the **power delivered to the motor**.

Fonctionnement global :

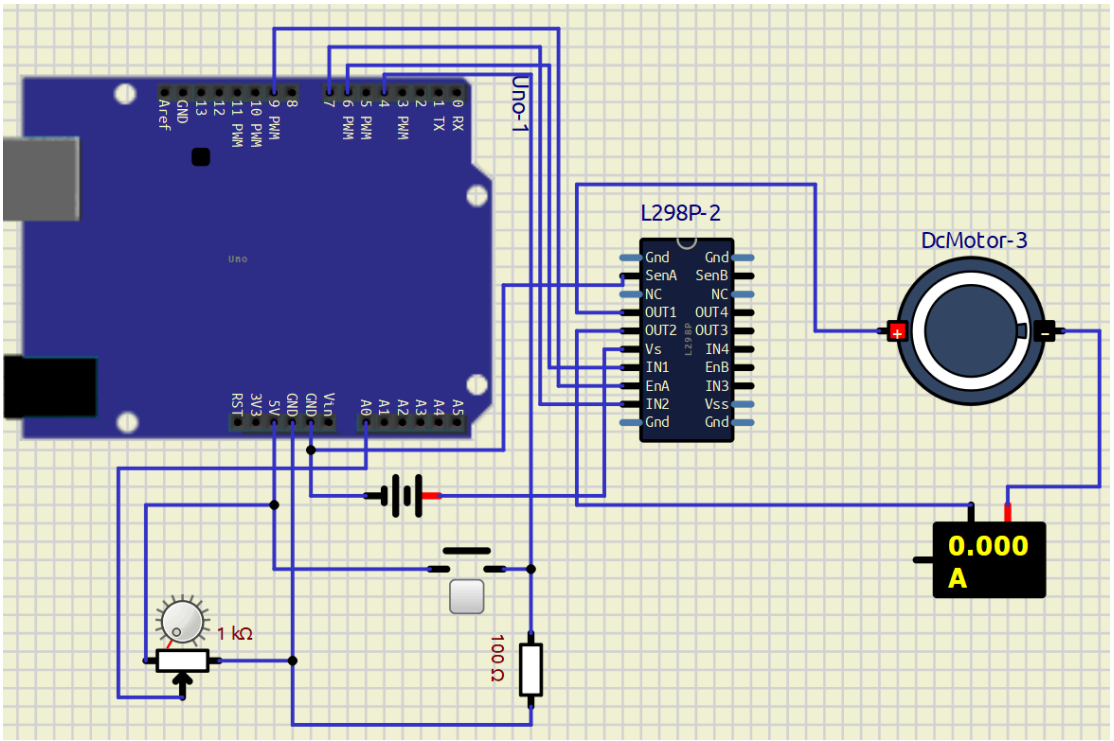
1. Le mode (manuel ou auto) est détecté, et le MUX oriente les signaux depuis Arduino ou pilote.
2. Les signaux de direction et de vitesse sont analysés.
3. La vitesse réelle est mesurée via un capteur.
4. L’erreur entre consigne et vitesse réelle est multipliée par K Gain.
5. Le résultat est transmis au module PWM, qui ajuste le moteur.
6. Le moteur agit (avance, recule ou change de direction).

Information and Energy chain

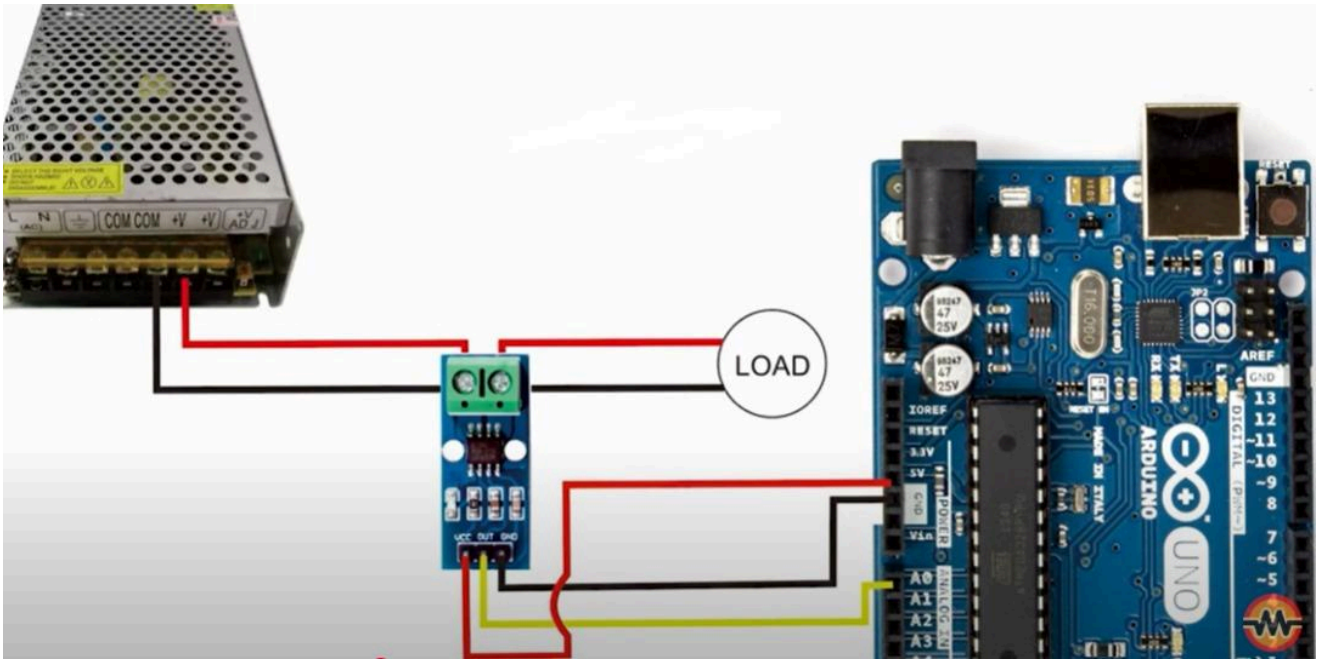


FINAL SIMULATION AND CODE

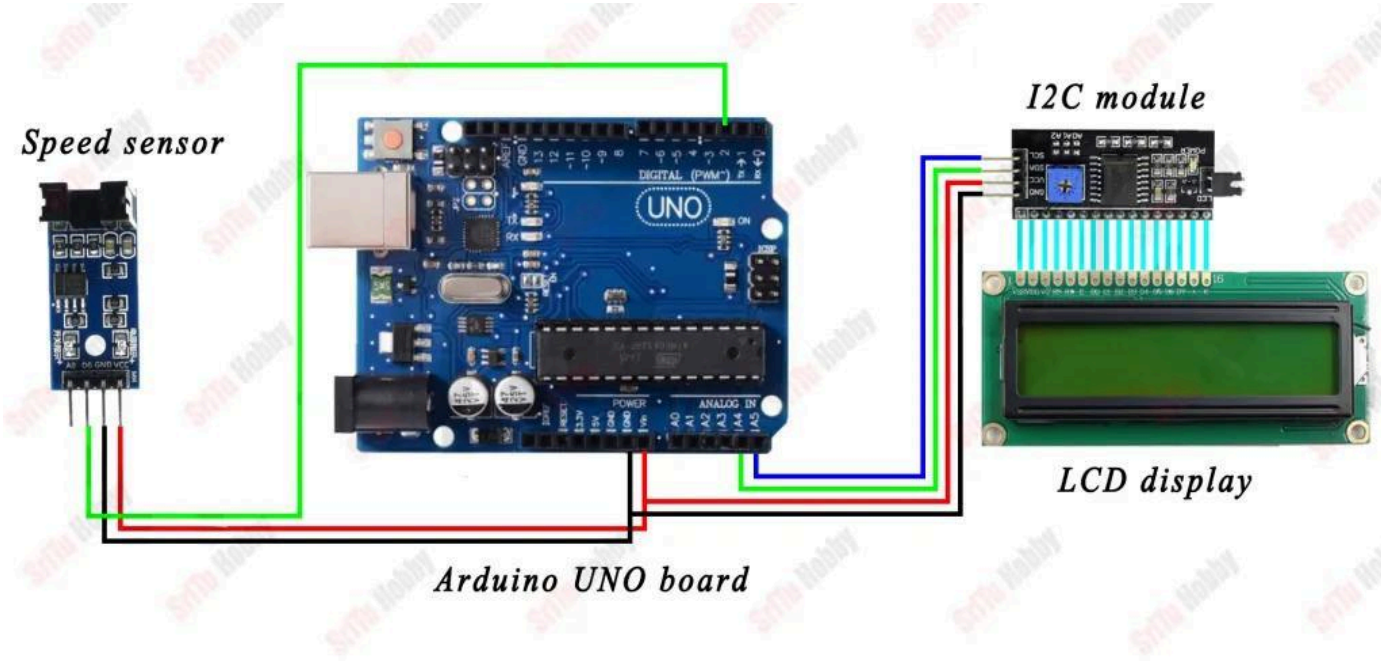
- H-Bridge, Dc motor connections



- **Current Sensor connection**



- **Speed sensor connection**



CODE

```
#include <LiquidCrystal.h>

// Initialisation de l'écran LCD (broches réaffectées)
LiquidCrystal lcd(3, 4, 5, 6, 10, 11);
```

```
// Définition des broches
#define POTENTIOMETER_PIN A0 // Potentiomètre
#define CURRENT_SENSOR_PIN A1 // Capteur ACS712
#define ENA 9              // Broche PWM pour contrôle de vitesse
#define IN1 7              // Direction du moteur
#define IN2 8              // Direction du moteur
#define SPEED_SENSOR_PIN 2   // Capteur infrarouge

// Variables pour le capteur ACS712
const float ACS712_SENSITIVITY = 0.185; // Sensibilité pour 5A
const int ACS712_OFFSET = 512;        // Valeur moyenne à 0A

// Variables pour le capteur de vitesse
volatile unsigned int pulseCount = 0;   // Nombre d'impulsions
float motorRPM = 0.0;               // RPM du moteur

void setup() {
  // Configuration des broches
  pinMode(ENA, OUTPUT);
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(SPEED_SENSOR_PIN, INPUT_PULLUP);

  // Configuration de l'interruption pour le capteur de vitesse
  attachInterrupt(digitalPinToInterrupt(SPEED_SENSOR_PIN), countPulse, FALLING);

  // Initialisation de l'écran LCD
  lcd.begin(16, 2); // Écran de 16 colonnes et 2 lignes
  lcd.print("Controle moteur");
  delay(2000);
  lcd.clear();

  // Initialisation du moteur
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);
  analogWrite(ENA, 0);
}

void loop() {
  // Lecture du potentiomètre et contrôle du moteur
  int potValue = analogRead(POTENTIOMETER_PIN); // Lecture du potentiomètre (0 à 1023)
  int motorSpeed = map(potValue, 0, 1023, 0, 255); // Conversion en 0-255
  analogWrite(ENA, motorSpeed);

  // Définir la direction du moteur
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);

  // Calcul de la vitesse du moteur
  motorRPM = calculateRPM();

  // Lecture de l'intensité du moteur
  float current = readCurrent();

  // Affichage sur l'écran LCD
  lcd.setCursor(0, 0);
  lcd.print("Vitesse: ");
```

```
  lcd.print(motorSpeed);
  lcd.print(" PWM");

  lcd.setCursor(0, 1);
  lcd.print("RPM: ");
  lcd.print(motorRPM, 1); // Une décimale
  lcd.print(" | I: ");
  lcd.print(current, 2); // Deux décimales
  lcd.print(" A");

  delay(500); // Pause pour actualisation
}

// Fonction pour compter les impulsions du capteur de vitesse
void countPulse() {
  pulseCount++;
}

// Fonction pour calculer la vitesse en RPM
float calculateRPM() {
  static unsigned long lastTime = 0;
  unsigned long currentTime = millis();
  unsigned int pulses;

  // Calculer le temps écoulé et les impulsions détectées
  noInterrupts();
  pulses = pulseCount;
  pulseCount = 0;
```

MeasureCurrent.ino

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x3F, 16, 2); // Adresse 0x3F, écran 16 colonnes, 2 lignes

#define enA 9
#define POTENTIOMETER_PIN A0
#define CURRENT_SENSOR_PIN A1
#define IN1 6     // Broche PWM pour IN1
#define IN2 7     // Broche PWM pour IN2
#define SPEED_SENSOR_PIN 2
#define DIRECTION_BUTTON_PIN 4  // Bouton pour changer de sens

const float ACS712_SENSITIVITY = 0.185;
const int ACS712_OFFSET = 512;

volatile unsigned int pulseCount = 0;
float motorRPM = 0.0;

bool motorDirection = true;           // true = avant, false = arrière
bool lastButtonState = HIGH;
```

MeasureCurrent.ino

```
26 void setup() {
27   pinMode(enA, OUTPUT);
28   pinMode(IN1, OUTPUT);
29   pinMode(IN2, OUTPUT);
30   pinMode(SPEED_SENSOR_PIN, INPUT_PULLUP);
31   pinMode(DIRECTION_BUTTON_PIN, INPUT_PULLUP);
32
33   attachInterrupt(digitalPinToInterrupt(SPEED_SENSOR_PIN), countPulse, FALLING);
34
35   lcd.init();
36   lcd.backlight();
37   lcd.print("Controle moteur");
38   delay(2000);
39   lcd.clear();
40
41   digitalWrite(IN1, LOW);
42   digitalWrite(IN2, LOW);
43 }
44
45 void loop() {
46   // Lecture et gestion du bouton (anti-rebond)
```

Output

MeasureCurrent.ino

```
46   // Lecture et gestion du bouton (anti-rebond)
47   bool reading = digitalRead(DIRECTION_BUTTON_PIN);
48   if (reading != lastButtonState) {
49     lastDebounceTime = millis();
50   }
51   if ((millis() - lastDebounceTime) > debounceDelay) {
52     if (reading != buttonState) {
53       buttonState = reading;
54       if (buttonState == LOW) {
55         motorDirection = !motorDirection; // inversion du sens
56       }
57     }
58   }
59   lastButtonState = reading;
60
61   // Lecture potentiomètre et calcul vitesse PWM
62   int potValue = analogRead(POTENTIOMETER_PIN);
63   int motorSpeed = map(potValue, 0, 1023, 0, 255);
64
65   analogWrite(enA, motorSpeed); // ✅ FIXED LINE: apply PWM to motor enable pin
66
67   // Application PWM sur IN1 ou IN2 selon direction
```

Output

MeasureCurrent.ino

```
67    // Application PWM sur IN1 ou IN2 selon direction
68    if (motorDirection) {
69      analogWrite(IN1, motorSpeed);
70      digitalWrite(IN2, LOW);
71    } else {
72      digitalWrite(IN1, LOW);
73      analogWrite(IN2, motorSpeed);
74    }
75
76    // Calcul vitesse moteur et courant
77    motorRPM = calculateRPM();
78    float current = readCurrent();
79
80    // Affichage LCD
81    lcd.setCursor(0, 0);
82    lcd.print("Speed:");
83    lcd.print(motorSpeed);
84    lcd.print("PWM ");
85
86    lcd.setCursor(0, 1);
87    lcd.print("RPM:");
```

MeasureCurrent.ino

```
86    lcd.setCursor(0, 1);
87    lcd.print("RPM:");
88    lcd.print(motorRPM, 1);
89    lcd.print(" I:");
90    lcd.print(current, 2);
91    lcd.print("A ");
92
93    delay(500);
94  }
95
96  void countPulse() {
97    pulseCount++;
98  }
99
100 float calculateRPM() {
101   static unsigned long lastTime = 0;
102   unsigned long currentTime = millis();
103   unsigned int pulses;
104
105   noInterrupts();
106   pulses = pulseCount;
```

MeasureCurrent.ino

```
105     noInterrupts();
106     pulses = pulseCount;
107     pulseCount = 0;
108     interrupts();
109
110     unsigned long timeElapsed = currentTime - lastTime;
111     lastTime = currentTime;
112
113     if (timeElapsed > 0) {
114       // 20 impulsions par tour (à ajuster selon capteur)
115       return (pulses / 20.0) * (60000.0 / timeElapsed);
116     } else {
117       return 0.0;
118     }
119   }
120
121   float readCurrent() {
122     int sensorValue = analogRead(CURRENT_SENSOR_PIN);
123     return (sensorValue - ACS712_OFFSET) * (5.0 / 1024.0) / ACS712_SENSITIVITY;
124   }
```